

Lab2: Sol SDK Workshop

Edan Chen

edan.chen@ganzin.com.tw

Outline

- Introduction of Sol SDK
- Lab0: Sol SDK Installation
- Lab1: Connect PC to Sol Glasses
- Case Study Demo

Background of Sol SDK



- Slow developing process
- Complex environment
- Steep learning curve

vs.



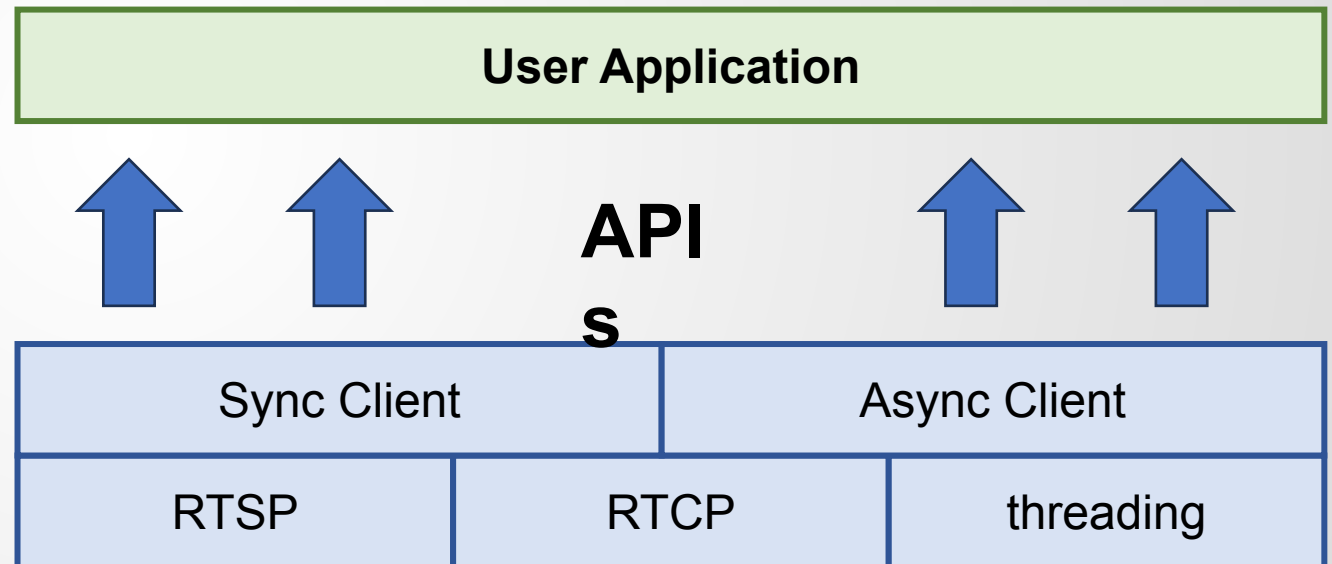
- Everyone knows how to use
- TOP programming language in the world!
- Many other packages can be integrated (PsychoPy, OpenAI API, etc.)



Fast eye tracking application development!

Architecture of Sol SDK

- Communication protocol: RTSP (Real Time Streaming Protocol), RTCP (Real-time Transport Control Protocol)
- Multi-threading: Python threading library
- Provides two kinds of clients
 - Sync Client
 - Easy to program
 - Good start point
 - Poor performance due to blocking
 - Async Client
 - More complicate
 - More real-time demonstration
- Let's learn the Sol SDK by going through the example code!!!



Lab0: Sol SDK Installation

Installation

- Install Python3.12

- win10

<https://www.microsoft.com/store/productId/9NCVDN91XZQP?ocid=pdpshare>

- Mac

\$ brew install python@3.12

- Download Sol SDK (v1.2.2)

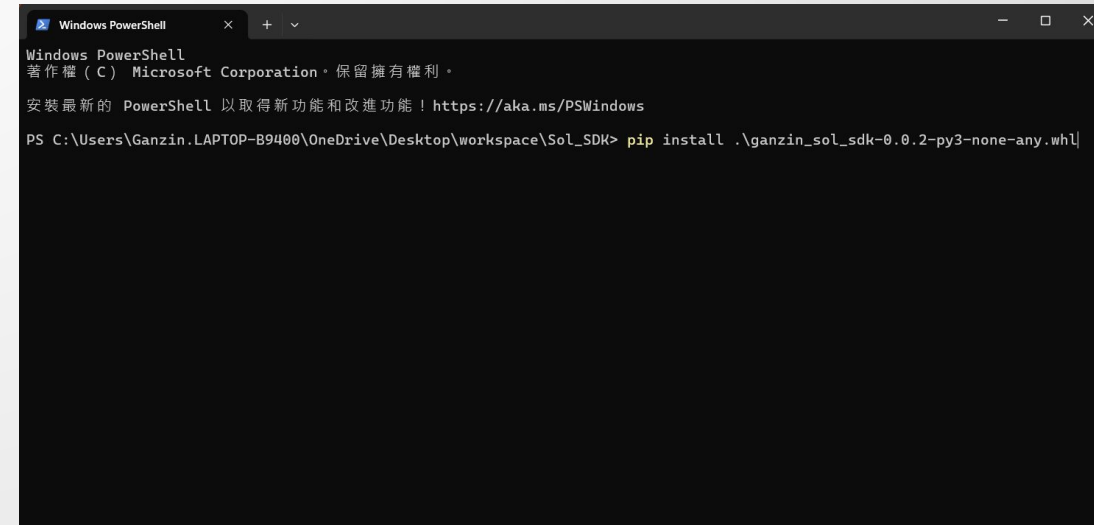
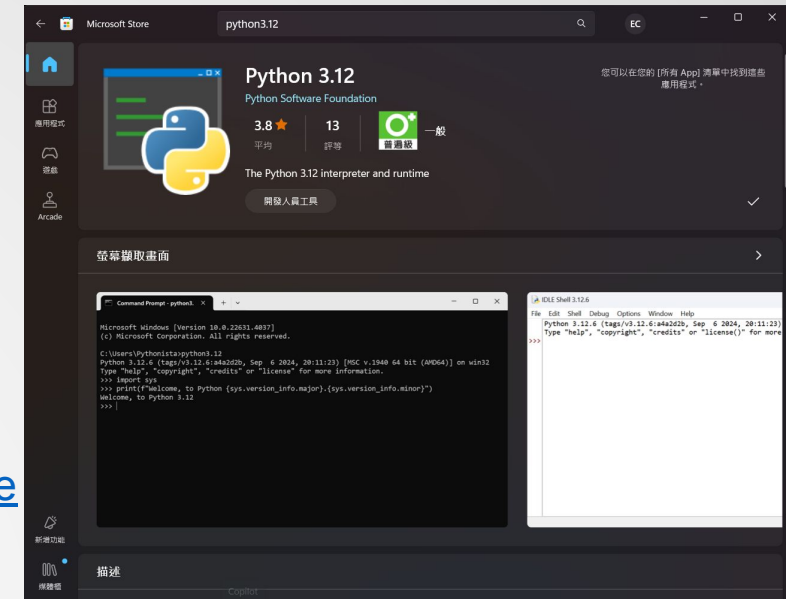
- whl: <https://drive.google.com/file/d/1RBcuDe8gsKRCRIWPhi93mX9nXdn4hYgO/view?usp=sharing>
 - examples.zip: https://drive.google.com/file/d/1_uN0Wd2OZF26yL34OH5KcF9aq-_qpqrF/view?usp=sharing
 - docs: <https://drive.google.com/file/d/1DTEAKEIbJQg4UkSn62f5dICF50cQN1Nb/view?usp=sharing>

- Install SDK

- In the Sol SDK directory
 - Right click to open the powershell (or terminal)
 - *\$ python3 -m pip install .\ganzin_sol_sdk-1.0.1-py3-none-any.whl*

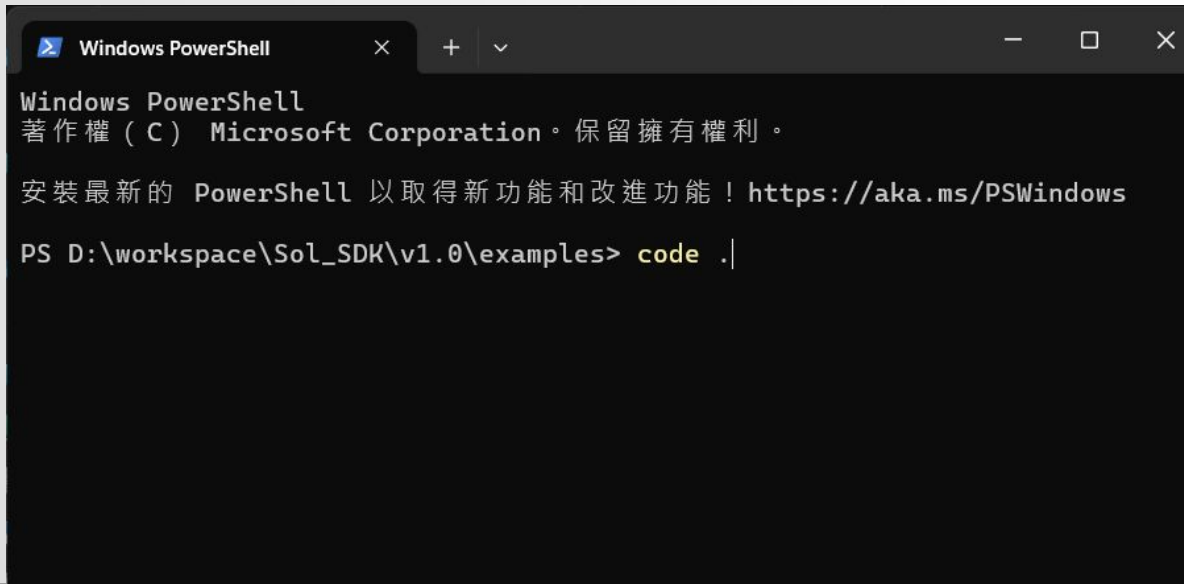
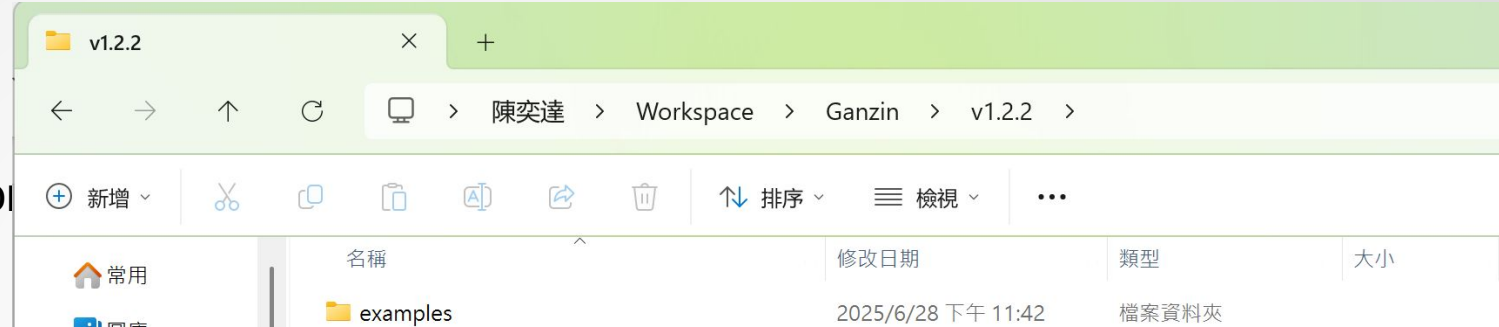
- Install other packages will used later

- *\$ python3 -m pip install requests opencv-python pyaudio*

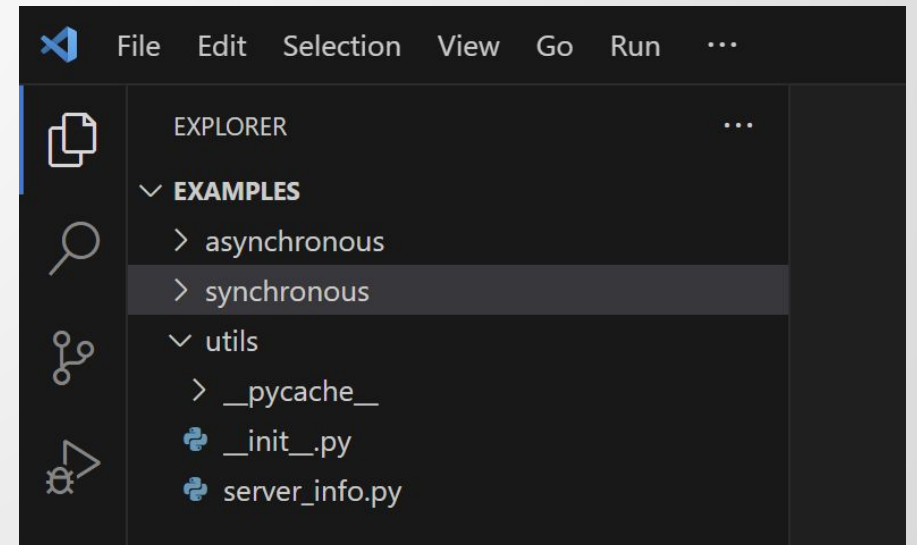


Dive Into Examples

- Extract examples.zip
 - Check the *examples* directory exist
- Open vscode in the examples directory
 - Go into the examples directory
 - Right click and open powershell
 - `$ code .`



- Check the hierarchy in vscode



Lab1: Connect PC to Sol Glasses

Previewing the gaze video

Setup Environment (1/2)

- Connect Sol Glasses's phone and your PC under the same AP (e.g., your own hotspot)

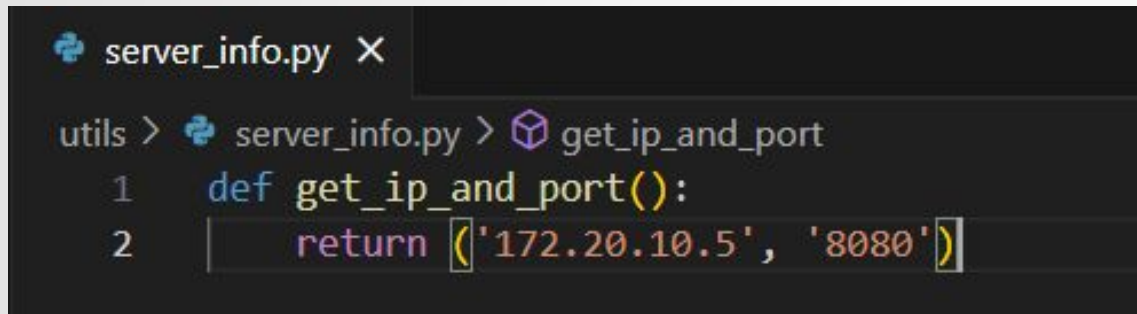


- Open the Android record tool and go into the preview window and click the computer icon
 - Get the IP and Port from the phone screen



Setup Environment (2/2)

- Fill the ip and port into the *utils/server_info.py*

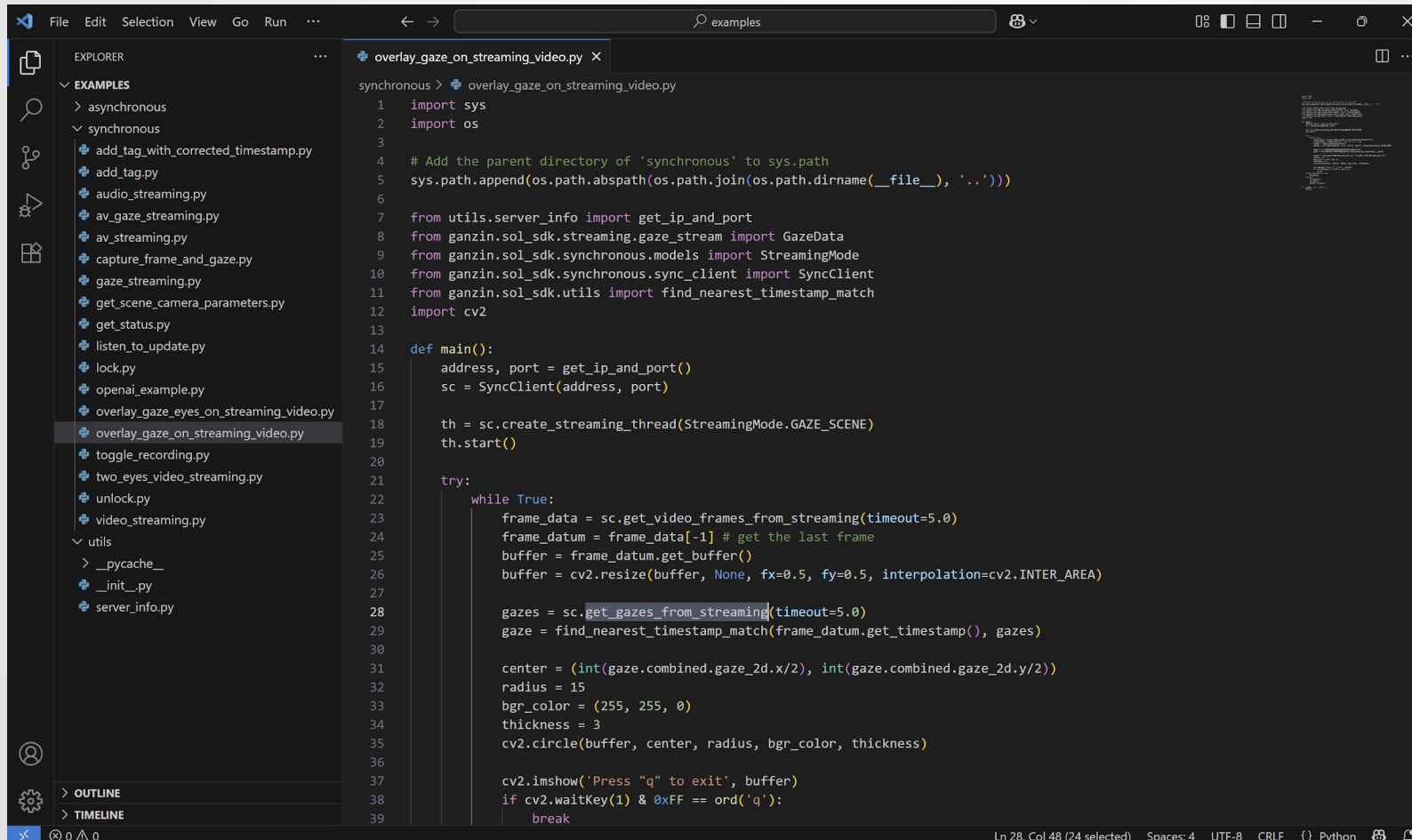


```
server_info.py X  
utils > server_info.py > get_ip_and_port  
1 def get_ip_and_port():  
2     return ('172.20.10.5', '8080')
```

- Please notice that when using the Sol SDK, the Android record tool needs to stay in the **preview window**

Dive into the Gaze Video Example

- Open the *synchronous/overlay_gaze_on_streaming_video.py* example



The screenshot shows a code editor with a dark theme. On the left, the 'EXPLORER' sidebar displays a file tree under 'EXAMPLES'. The 'synchronous' folder is expanded, showing various Python files. The file 'overlay_gaze_on_streaming_video.py' is selected and highlighted. The main editor area displays the code for this file. The code imports 'sys' and 'os', adds the parent directory to 'sys.path', and imports several modules from 'ganzin.sol_sdk'. It defines a 'main()' function that sets up a 'SyncClient', creates a streaming thread, and enters a loop to fetch video frames and gaze data, processing them to draw a circle on the video frame. The status bar at the bottom indicates 'Ln 28, Col 48 (24 selected)'.

```
1 import sys
2 import os
3
4 # Add the parent directory of 'synchronous' to sys.path
5 sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))
6
7 from utils.server_info import get_ip_and_port
8 from ganzin.sol_sdk.streaming.gaze_stream import GazeData
9 from ganzin.sol_sdk.synchronous.models import StreamingMode
10 from ganzin.sol_sdk.synchronous.sync_client import SyncClient
11 from ganzin.sol_sdk.utils import find_nearest_timestamp_match
12 import cv2
13
14 def main():
15     address, port = get_ip_and_port()
16     sc = SyncClient(address, port)
17
18     th = sc.create_streaming_thread(StreamingMode.GAZE_SCENE)
19     th.start()
20
21     try:
22         while True:
23             frame_data = sc.get_video_frames_from_streaming(timeout=5.0)
24             frame_datum = frame_data[-1] # get the last frame
25             buffer = frame_datum.get_buffer()
26             buffer = cv2.resize(buffer, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)
27
28             gazes = sc.get_gazes_from_streaming(timeout=5.0)
29             gaze = find_nearest_timestamp_match(frame_datum.get_timestamp(), gazes)
30
31             center = (int(gaze.combined.gaze_2d.x/2), int(gaze.combined.gaze_2d.y/2))
32             radius = 15
33             bgr_color = (255, 255, 0)
34             thickness = 3
35             cv2.circle(buffer, center, radius, bgr_color, thickness)
36
37             cv2.imshow('Press "q" to exit', buffer)
38             if cv2.waitKey(1) & 0xFF == ord('q'):
39                 break
```

Dive into the Code (1/4)

- Import modules
 - sys, os: to make the server_info importable
 - ganzin.sol_sdk: the sdk to remote control the Sol Glasses
 - cv2: visualize the result get from the Sol SDK

```
1  import sys
2  import os
3
4  # Add the parent directory of 'synchronous' to sys.path
5  sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))
6
7  from utils.server_info import get_ip_and_port
8  from ganzin.sol_sdk.streaming.gaze_stream import GazeData
9  from ganzin.sol_sdk.synchronous.models import StreamingMode
10 from ganzin.sol_sdk.synchronous.sync_client import SyncClient
11 from ganzin.sol_sdk.utils import find_nearest_timestamp_match
12 import cv2
```

Dive into the Code (2/4)

- Sync Client setup
 - Get the address and port from `get_ip_and_port()`
 - Create a `SyncClient` using the address and port
 - Create a *streaming thread* for the Sync Client
 - Start the thread

```
14  def main():
15      address, port = get_ip_and_port()
16      sc = SyncClient(address, port)
17
18      th = sc.create_streaming_thread(StreamingMode.GAZE_SCENE)
19      th.start()
```


Dive into the Code (3/4)

- Getting the gazes and the frames
 - Create an infinite while loop to keep getting the gazes and frames
 - Get the frame data from `get_video_frames_from_streaming()`
 - Resize the buffer as the resolution of the Sol Glasses scene camera is 1328x1200 (exceed 1080)
 - Get the gazes data from `get_gazes_from_streaming()`
 - Get the exactly gaze using the timestamp of the frame

```
21     try:
22         while True:
23             frame_data = sc.get_video_frames_from_streaming(timeout=5.0)
24             frame_datum = frame_data[-1] # get the last frame
25             buffer = frame_datum.get_buffer()
26             buffer = cv2.resize(buffer, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)
27
28             gazes = sc.get_gazes_from_streaming(timeout=5.0)
29             gaze = find_nearest_timestamp_match(frame_datum.get_timestamp(), gazes)
```

Dive into the Code (4/4)

- Show the result using cv2
 - Calculate the center of the circle
 - Set the circle's radius, color, and thickness
 - Draw the circle using `cv2.circle()`
 - Show the result using `cv2.imshow()`

```
31     center = (int(gaze.combined.gaze_2d.x/2), int(gaze.combined.gaze_2d.y/2))
32     radius = 15
33     bgr_color = (255, 255, 0)
34     thickness = 3
35     cv2.circle(buffer, center, radius, bgr_color, thickness)
36
37     cv2.imshow('Press "q" to exit', buffer)
38     if cv2.waitKey(1) & 0xFF == ord('q'):
39         break
```

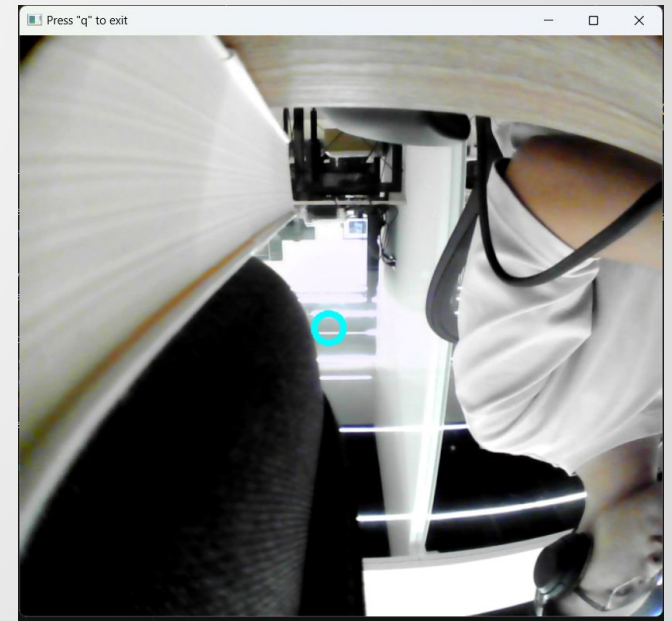
Run the Example

- Open terminal from the vscode

```
overlay_gaze_on_streaming_video.py X
synchronous > overlay_gaze_on_streaming_video.py > main
14 def main():
16     sc = SyncClient(address, port)
17
18     th = sc.create_streaming_thread(StreamingMode.GAZE_SCENE)
19     th.start()

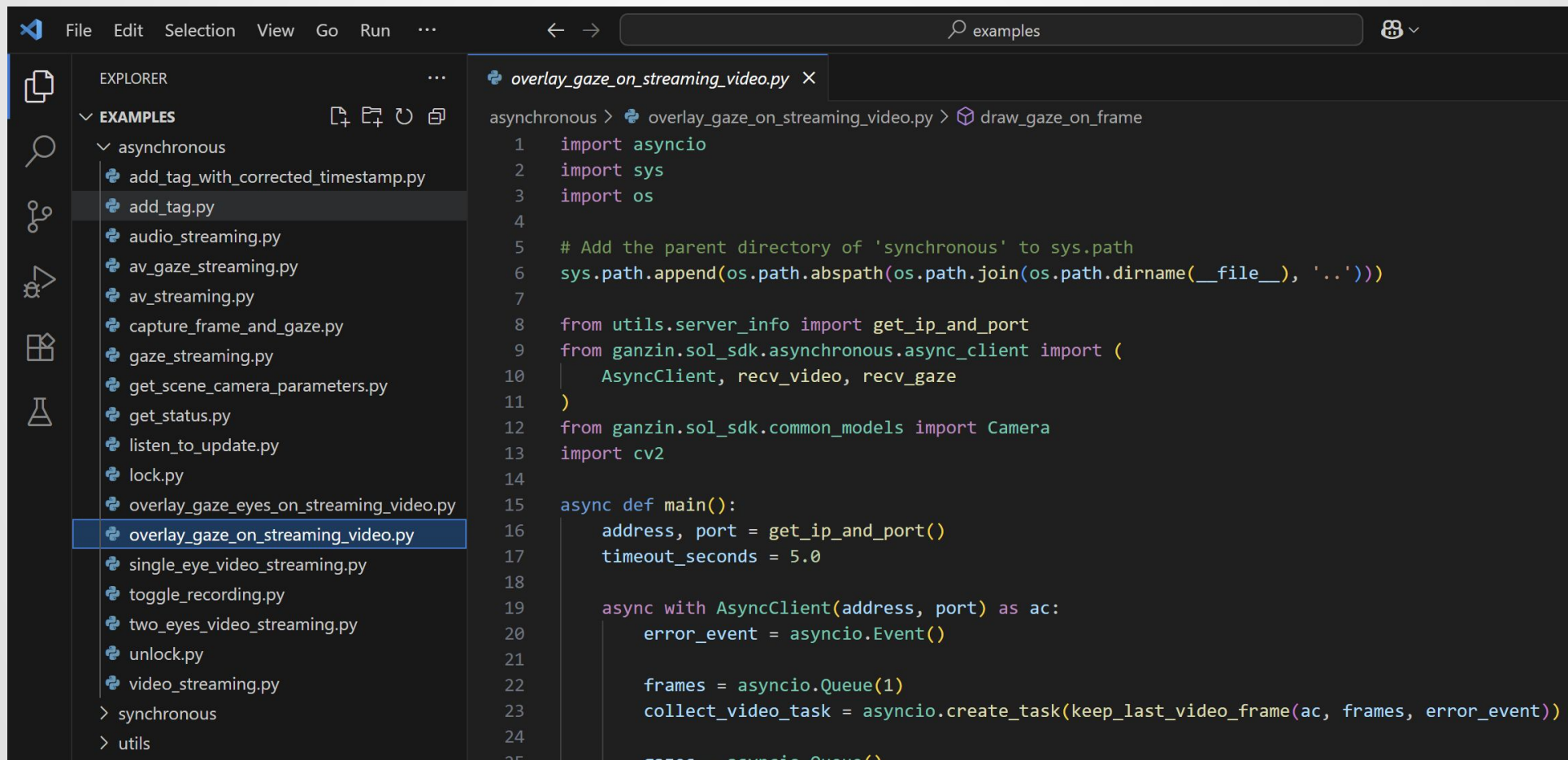
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS python3.12
PS C:\Users\edan8\Workspace\Ganzin\v1.2.2\examples> python3 .\synchronous\overlay_gaze_on_streaming_video.py
Note: Your SDK version is newer in terms of patch versions.
```

- `$ python3 .\synchronous\overlay_gaze_on_streaming_video.py`
 - Check whether the streaming window appear
 - Press 'q' to exit



Dive into the Async Counterpart

- Open the *asynchronous/overlay_gaze_on_streaming_video.py* example



The screenshot shows a code editor interface with a dark theme. On the left, the 'EXPLORER' sidebar is open, displaying a file tree under the 'EXAMPLES' folder. The 'asynchronous' subfolder is expanded, and the file 'overlay_gaze_on_streaming_video.py' is selected and highlighted in blue. The main editor area on the right shows the code for this file. The code is a Python script that uses the 'asyncio' library to handle asynchronous tasks. It includes imports for 'asyncio', 'sys', and 'os'. A comment indicates that the parent directory of 'synchronous' is added to 'sys.path'. The script then imports 'get_ip_and_port' from 'utils.server_info' and 'AsyncClient', 'recv_video', and 'recv_gaze' from 'ganzin.sol_sdk.asynchronous.async_client'. It also imports 'Camera' from 'ganzin.sol_sdk.common_models' and 'cv2'. The 'main' function is defined as an asynchronous function. It calls 'get_ip_and_port' to get an address and port, and sets a timeout of 5.0 seconds. It then creates an 'AsyncClient' instance and an 'Event' object. A queue 'frames' is created with a size of 1. An asynchronous task 'collect_video_task' is created using 'asyncio.create_task' to call 'keep_last_video_frame'. The code ends with a line that is partially cut off: 'gazes = asyncio.Queue()'.

```
1 import asyncio
2 import sys
3 import os
4
5 # Add the parent directory of 'synchronous' to sys.path
6 sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))
7
8 from utils.server_info import get_ip_and_port
9 from ganzin.sol_sdk.asynchronous.async_client import (
10     AsyncClient, recv_video, recv_gaze
11 )
12 from ganzin.sol_sdk.common_models import Camera
13 import cv2
14
15 async def main():
16     address, port = get_ip_and_port()
17     timeout_seconds = 5.0
18
19     async with AsyncClient(address, port) as ac:
20         error_event = asyncio.Event()
21
22         frames = asyncio.Queue(1)
23         collect_video_task = asyncio.create_task(keep_last_video_frame(ac, frames, error_event))
24
25         gazes = asyncio.Queue()
```

Dive into the Code (1/5)

- Module import
 - `asyncio`: modules for asynchronous programming, enables the execution of I/O-bound tasks without blocking the execution of other tasks.
 - `ganzin.sol_sdk`: import the `AsyncClient`

```
1  import asyncio
2  import sys
3  import os
4
5  # Add the parent directory of 'synchronous' to sys.path
6  sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))
7
8  from utils.server_info import get_ip_and_port
9  from ganzin.sol_sdk.asynchronous.async_client import (
10 |     AsyncClient, recv_video, recv_gaze
11 | )
12  from ganzin.sol_sdk.common_models import Camera
13  import cv2
```

Dive into the Code (2/5)

- Main function
 - Set address and port of AsyncClient, timeout_seconds for collect_gaze function (considering internet environment)
 - Create two queues: frames, gazes, for putting the frame/gaze into the queue
 - Create two tasks: collect_video_task, collect_gaze_task, for putting the frame get from the AsyncClient into the queue
 - Await for the *draw_gaze_on_frame()*, draw the frame and gaze when there is gaze/frame in the queues

```
15 async def main():
16     address, port = get_ip_and_port()
17     timeout_seconds = 5.0
18
19     async with AsyncClient(address, port) as ac:
20         error_event = asyncio.Event()
21
22         frames = asyncio.Queue(1)
23         collect_video_task = asyncio.create_task(keep_last_video_frame(ac, frames, error_event))
24
25         gazes = asyncio.Queue()
26         collect_gaze_task = asyncio.create_task(collect_gaze(ac, gazes, error_event, timeout_seconds))
27
28         try:
29             await draw_gaze_on_frame(frames, gazes, error_event, timeout_seconds)
30         finally:
31             collect_video_task.cancel()
32             collect_gaze_task.cancel()
```

Dive into the Code (3/5)

- *keep_last_video_frame()*
 - async for loop for getting frame from *recv_video()*
 - This will be blocked by the IO
 - Remove the item in queue if full ASAP, the queue has only one seat
 - Put the frame into the queue ASAP
- *collect_gaze()*
 - async for loop for getting gaze from *rec_gaze()*
 - put the gaze into the queue
 - The queue can preserve multiple gazes

```
34 async def keep_last_video_frame(ac: AsyncClient, queue: asyncio.Queue, error_event: asyncio.Event) -> None:
35     async for frame in recv_video(ac, Camera.SCENE):
36         if error_event.is_set():
37             break
38
39         if queue.full():
40             queue.get_nowait()
41             queue.put_nowait(frame)
42
43 async def collect_gaze(ac: AsyncClient, queue: asyncio.Queue, error_event: asyncio.Event, timeout) -> None:
44     try:
45         async for gaze in rec_gaze(ac):
46             if error_event.is_set():
47                 break
48
49             await asyncio.wait_for(queue.put(gaze), timeout=timeout)
50     except Exception as e:
51         error_event.set()
```


Dive into the Code (4/5)

- `draw_gaze_on_frame()`
 - Get the frame from the `frame_queue`
 - Get the gaze from the `gaze_queue`
 - Resize frame buffer and draw the gaze on the frame using `cv2`

```
53  async def draw_gaze_on_frame(frame_queue, gazes, error_event: asyncio.Event, timeout):
54      while not error_event.is_set():
55          frame = await get_video_frame(frame_queue, timeout)
56          gaze = await find_gaze_near_frame(gazes, frame.get_timestamp(), timeout)
57          frame_buffer = frame.get_buffer()
58          frame_buffer = cv2.resize(frame_buffer, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)
59
60          center = (int(gaze.combined.gaze_2d.x/2), int(gaze.combined.gaze_2d.y/2))
61          radius = 15
62          bgr_color = (255, 255, 0)
63          thickness = 3
64          cv2.circle(frame_buffer, center, radius, bgr_color, thickness)
65
66          cv2.imshow('Press "q" to exit', frame_buffer)
67          if cv2.waitKey(1) & 0xFF == ord('q'):
68              return
```

Dive into the Code (5/5)

- *get_video_frame()*
 - Async function for getting the frame out of the queue
- *find_gaze_near_frame()*
 - Async function for finding the most appropriate gaze in the gaze queue

```
70  async def get_video_frame(queue, timeout):
71      return await asyncio.wait_for(queue.get(), timeout=timeout)
72
73  async def find_gaze_near_frame(queue, timestamp, timeout):
74      item = await asyncio.wait_for(queue.get(), timeout=timeout)
75      if item.get_timestamp() > timestamp:
76          return item
77
78      while True:
79          if queue.empty():
80              return item
81          else:
82              next_item = queue.get_nowait()
83              if next_item.get_timestamp() > timestamp:
84                  return next_item
85              item = next_item
```

Run the Example

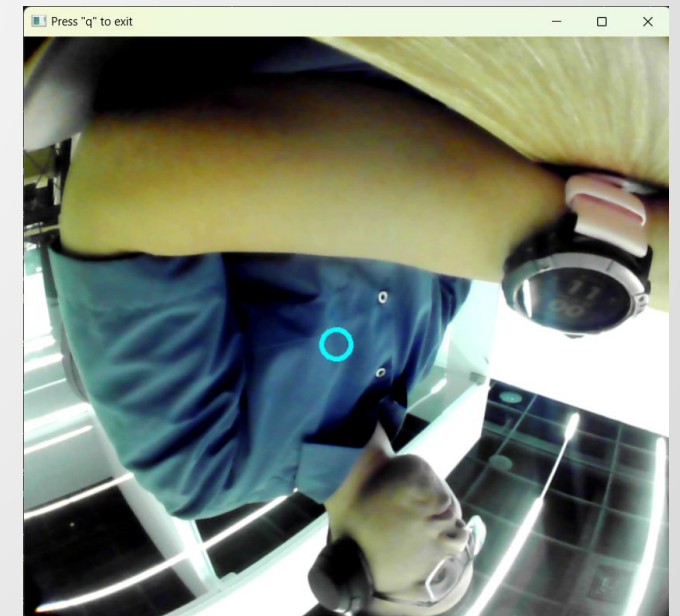
- Open terminal from the vscode

```
overlay_gaze_on_streaming_video.py X
asynchronous > overlay_gaze_on_streaming_video.py > ...
53  async def draw_gaze_on_frame(frame_queue, gazes, error_event: asyncio.Event, timeout):
57      frame_buffer = frame.get_buffer()
58      frame_buffer = cv2.resize(frame_buffer, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)
59
60      center = (int(gaze.combined.gaze_2d.x/2), int(gaze.combined.gaze_2d.v/2))

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  python3.12 ⚠

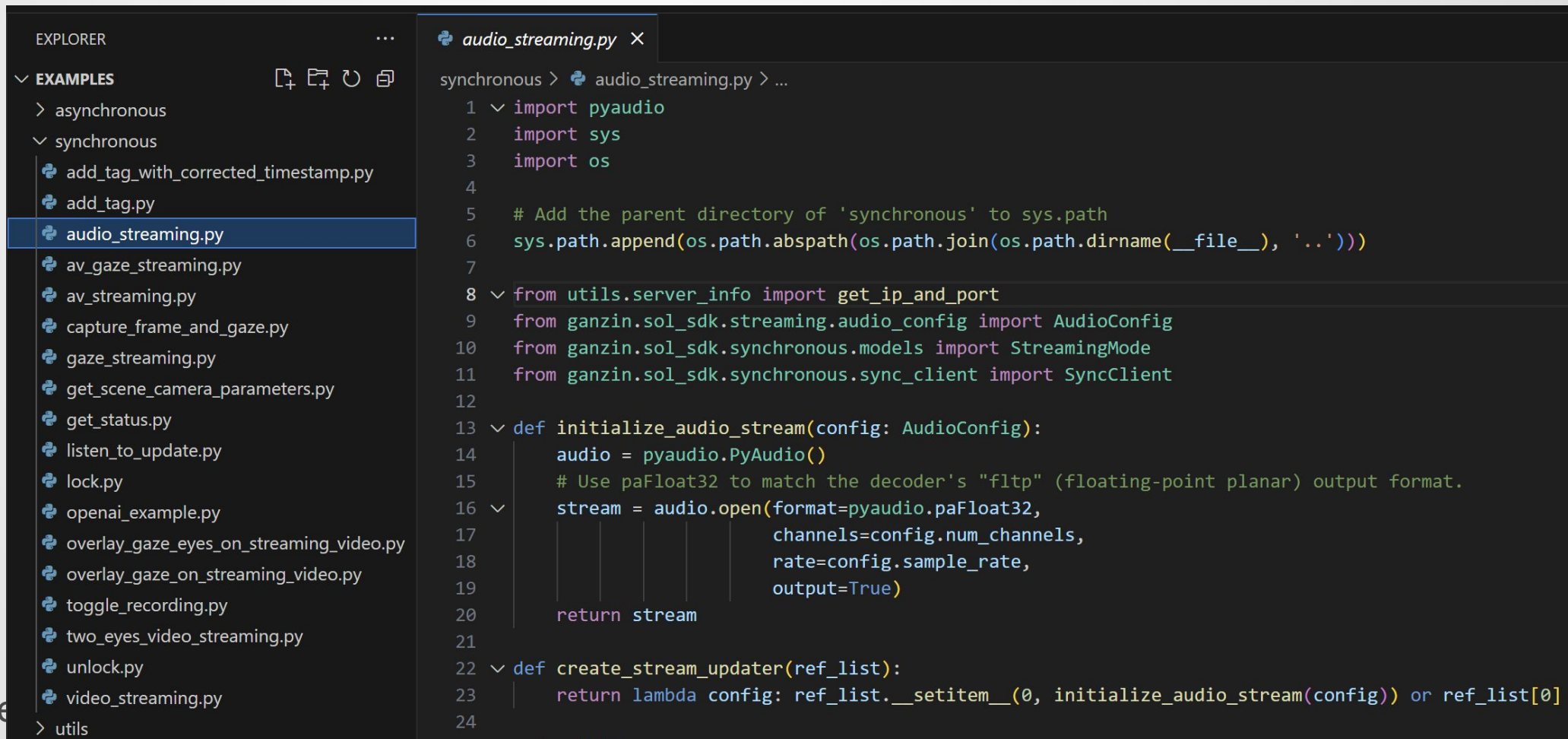
'NoneType' object is not iterable
PS C:\Users\edan8\Workspace\Ganzin\v1.2.2\examples> python3 .\asynchronous\overlay_gaze_on_streaming_video.py
Note: Your SDK version is newer in terms of patch versions.
```

- `$ python3 .\asynchronous\overlay_gaze_on_streaming_video.py`
 - Check whether the streaming window appear
 - Press 'q' to exit
 - Same result from the synchronous counterpart



Dive into the Audio Example

- Open the *synchronous/audio_streaming.py* example



The screenshot shows a code editor with a dark theme. On the left, the 'EXPLORER' sidebar is open, showing a file tree under 'EXAMPLES'. The 'synchronous' folder is expanded, and 'audio_streaming.py' is selected. The main editor area shows the code for 'audio_streaming.py'. The code includes imports for 'pyaudio', 'sys', and 'os', followed by a comment and a path manipulation line. It then imports 'get_ip_and_port' from 'utils.server_info', 'AudioConfig' from 'ganzin.sol_sdk.streaming.audio_config', 'StreamingMode' from 'ganzin.sol_sdk.synchronous.models', and 'SyncClient' from 'ganzin.sol_sdk.synchronous.sync_client'. The code defines two functions: 'initialize_audio_stream' which creates a PyAudio object, opens a stream with specific format and channel settings, and returns it; and 'create_stream_updater' which returns a lambda function that calls 'initialize_audio_stream' and updates a reference list.

```
EXPLORER
EXAMPLES
  asynchronous
  synchronous
    add_tag_with_corrected_timestamp.py
    add_tag.py
    audio_streaming.py
    av_gaze_streaming.py
    av_streaming.py
    capture_frame_and_gaze.py
    gaze_streaming.py
    get_scene_camera_parameters.py
    get_status.py
    listen_to_update.py
    lock.py
    openai_example.py
    overlay_gaze_eyes_on_streaming_video.py
    overlay_gaze_on_streaming_video.py
    toggle_recording.py
    two_eyes_video_streaming.py
    unlock.py
    video_streaming.py
  utils

audio_streaming.py
synchronous > audio_streaming.py > ...
1  import pyaudio
2  import sys
3  import os
4
5  # Add the parent directory of 'synchronous' to sys.path
6  sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))
7
8  from utils.server_info import get_ip_and_port
9  from ganzin.sol_sdk.streaming.audio_config import AudioConfig
10 from ganzin.sol_sdk.synchronous.models import StreamingMode
11 from ganzin.sol_sdk.synchronous.sync_client import SyncClient
12
13 def initialize_audio_stream(config: AudioConfig):
14     audio = pyaudio.PyAudio()
15     # Use paFloat32 to match the decoder's "fltp" (floating-point planar) output format.
16     stream = audio.open(format=pyaudio.paFloat32,
17                         channels=config.num_channels,
18                         rate=config.sample_rate,
19                         output=True)
20     return stream
21
22 def create_stream_updater(ref_list):
23     return lambda config: ref_list.__setitem__(0, initialize_audio_stream(config)) or ref_list[0]
24
```

Dive into the Code (1/3)

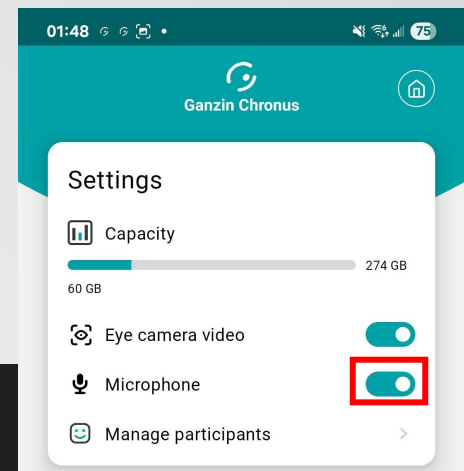
- Module import
 - pyaudio: modules to process the audio get from Sol Glasses
 - ganzin.sol_sdk: import the SyncClient and AudioConfig

```
1  import pyaudio
2  import sys
3  import os
4
5  # Add the parent directory of 'synchronous' to sys.path
6  sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))
7
8  from utils.server_info import get_ip_and_port
9  from ganzin.sol_sdk.streaming.audio_config import AudioConfig
10 from ganzin.sol_sdk.synchronous.models import StreamingMode
11 from ganzin.sol_sdk.synchronous.sync_client import SyncClient
```

Dive into the Code (2/3)

- Main function
 - Set address and port of AsyncClient
 - Check the audio_enabled label
 - Need to enable audio in Chronus
 - Create a steam to get the audio data
 - Setup the audio callback function
 - Initialize audio stream when new audio config is received
 - Create steaming thread for pure audio
 - In while loop
 - Get audio frame
 - Write the pcm of the frame to the audio stream and play it out with PyAudio

```
25 def main():
26     address, port = get_ip_and_port()
27     sc = SyncClient(address, port)
28     if not sc.get_status().audio_enabled:
29         print("Warning: Please enable 'Microphone' in the server settings.")
30         return
31
32     streams = [None]
33
34     # Create a callback function that initializes the audio stream when
35     # new audio configuration is received. This function updates the stream
36     # reference in the streams list.
37     update_audio_stream = create_stream_updater(streams)
38
39     print("Audio playback started. Press Ctrl+C to stop.")
40
41     th = sc.create_streaming_thread(StreamingMode.AUDIO, update_audio_stream)
42     th.start()
43
44     try:
45         while True:
46             frame_data = sc.get_audio_frames_from_streaming(timeout=5.0)
47             if not frame_data:
48                 continue
49             for sample in frame_data:
50                 streams[0].write(sample.get_pcm())
51
52     except KeyboardInterrupt: # Press Ctrl-C to stop
53         pass
```



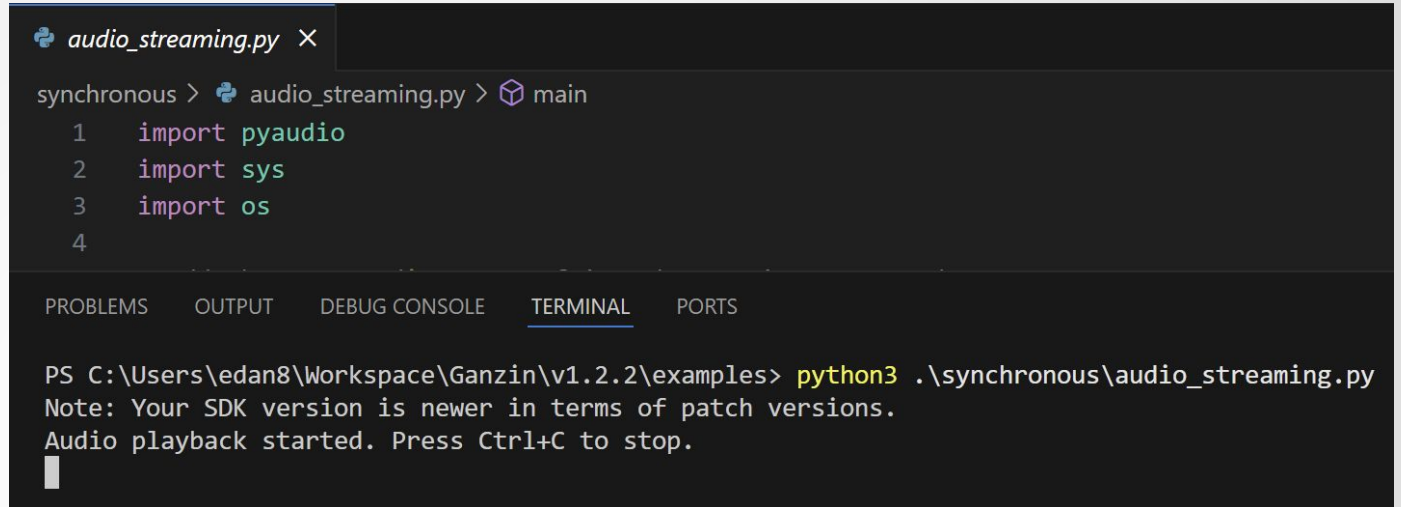
Dive into the Code (3/3)

- *initialize_audio_stream()*
 - Given an *AudioConfig* (sample rate, channels), it:
 - Instantiates a *pyaudio.PyAudio()* object
 - Opens a stream with *paFloat32* format matching the decoder's "fltp"
 - Returns the ready-to-write PyAudio stream
- *create_stream_updater()*
 - Your helper that takes a one-element list (*streams*) and returns a callback fn (config: *AudioConfig*) →

```
13 def initialize_audio_stream(config: AudioConfig):
14     audio = pyaudio.PyAudio()
15     # Use paFloat32 to match the decoder's "fltp" (floating-point planar) output format.
16     stream = audio.open(format=pyaudio.paFloat32,
17                          channels=config.num_channels,
18                          rate=config.sample_rate,
19                          output=True)
20     return stream
21
22 def create_stream_updater(ref_list):
23     return lambda config: ref_list.__setitem__(0, initialize_audio_stream(config)) or ref_list[0]
```

Run the Example

- Open terminal from the vscode



The screenshot shows a VS Code editor with a file named `audio_streaming.py` open. The terminal window is active, showing the command `python3 .\synchronous\audio_streaming.py` being executed. The output indicates that the SDK version is newer than the patch versions and that audio playback has started. The terminal also shows the first few lines of the Python script: `import pyaudio`, `import sys`, and `import os`.

```
audio_streaming.py X
synchronous > audio_streaming.py > main
1 import pyaudio
2 import sys
3 import os
4
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\edan8\Workspace\Ganzin\v1.2.2\examples> python3 .\synchronous\audio_streaming.py
Note: Your SDK version is newer in terms of patch versions.
Audio playback started. Press Ctrl+C to stop.
```

- `$ python3 .\asynchronous\audio_streaming.py`
 - Check whether you can hear the sound recorded by Sol's microphone
 - Press 'ctrl-c' to exit

Case Study Demo

Q&A